

Panduan Persiapan dan Eksekusi

FASE 0: PERSIAPAN DOKUMEN KONTEKS (Anti-Halusinasi)

Sebelum mengetik satu baris pun kode atau *prompt* ke AI, Anda harus menyiapkan "Single Source of Truth" (SSOT). Halusinasi AI terjadi karena konteksnya kabur.

Yang harus disiapkan:

1. Dokumen Arsitektur Sistem (System Architecture Document): Gambar bagaimana alur data dari Flutter -> Local DB (Isar) -> Sync Engine -> Backend API -> PostgreSQL.
2. Skema Basis Data (ERD & Schema): Tentukan tabel `users`, `roles`, `blocks`, `tph` (Tempat Pengumpulan Hasil), `harvest_logs`. Jangan biarkan AI mendesain skema DB dari nol.
3. Matriks RBAC & Rumusan Algoritma: Tuliskan secara matematis rumusan resolusi konflik: $\text{Priority Score} = (\text{Role Weight} \times 1.000.000) + \text{Timestamp}$.
4. Dokumen API Contract (OpenAPI/Swagger): Tentukan endpoint apa saja yang ada (misal: `POST /api/v1/sync`, `GET /api/v1/blocks`).
5. Setup Repositori Git: Buat monorepo atau multi-repo (misal: `agrisionc-mobile`, `agrisionc-backend`).

FASE 1: AI-ASSISTED SCAFFOLDING & SETUP

Pada tahap ini, gunakan AI hanya untuk mempercepat pembuatan kerangka (boilerplate), bukan logika inti.

Step 1: Backend Setup (NestJS/Laravel + PostgreSQL/PostGIS)

- Prompt AI: "Buatkan struktur proyek awal NestJS dengan modul *Users*, *Blocks*, dan *Sync*. Sertakan konfigurasi TypeORM untuk PostgreSQL dengan ekstensi PostGIS. Jangan buat logika bisnis dulu, hanya kerangka folder, DTO, dan Entity."
- Eksekusi Manusia (Anda): Validasi struktur folder. Sesuaikan tipe data geospatial (Polygon/Point) di Entity dengan standar PostGIS.

Step 2: Mobile Setup (Flutter + Isar DB)

- Prompt AI: "Setup proyek Flutter dengan arsitektur Clean Architecture (Domain, Data, Presentation layer). Tambahkan dependensi: `isar`, `isar_flutter_libs`, `path_provider`, `dio`, dan `flutter_bloc`."
- Eksekusi Manusia: Pastikan versi package compatible. Buat model Isar DB untuk `HarvestLog` secara manual karena ini adalah inti dari offline-first.

FASE 2: PENGEMBANGAN CORE SYSTEM & SYNC ENGINE

PERINGATAN: Ini adalah bagian paling krusial. Jangan pernah menyuruh AI menulis *Sync Engine* dalam satu prompt besar. Pecah menjadi modular.

Step 1: Implementasi RBAC 5 Jenjang

- Persiapan: Buat *enum* atau *constant* untuk Role Weight (Manager=5, Askep=4, dst).
- Prompt AI: "Buatkan middleware di NestJS untuk validasi JWT dan cek role pengguna. Gunakan decorator `@Roles('manager', 'askep')`. Tolak akses jika tidak sesuai."
- Eksekusi Manusia: Lakukan unit testing masing-masing role secara manual menggunakan Postman/Bruno.

Step 2: Membangun Algoritma Domain-Specific Conflict Resolution

- Persiapan: Tulis pseudocode di kertas atau Notion

```
IF incoming_data.priority_score > existing_data.priority_score THEN
```

```
    UPDATE existing_data
```

```
ELSE IF priority_score == priority_score THEN
```

```
    // Fallback ke timestamp milidetik
```

```
    IF incoming_data.timestamp > existing_data.timestamp THEN UPDATE
```

- Prompt AI: "Saya punya pseudocode [masukkan pseudocode di atas]. Tolong translasikan ke fungsi TypeScript di NestJS Service. Pastikan menggunakan tipe data `BigInt` untuk Timestamp milidetik agar tidak hilang presisinya."
- Eksekusi Manusia: Uji dengan skenario mock-up: Krani input di menit ke-1, Asisten edit di menit ke-2 (saat offline), lalu sinkronisasi. Pastikan data Asisten yang menang.

Step 3: Local Sync Engine di Flutter (Store-and-Forward)

- Prompt AI: "Buatkan sebuah `SyncService` di Flutter menggunakan Bloc. Service ini harus mengecek koneksi internet setiap 30 detik. Jika online, ambil data antrian dari koleksi `pending_syncs` di Isar DB, kirim via Dio dengan metode POST ke endpoint `/sync`, dan hapus dari antrian jika response 200."
- Eksekusi Manusia: Implementasi `interceptor` di Dio untuk menangani error 409 (Conflict) dari backend, agar Flutter tahu kapan harus mengirim ulang data dengan priority score.

FASE 3: UI/UX INTEGRATION & GEOSPATIAL (Bulan 3)

Step 1: Geospatial Tagging (EUDR/Traceability)

- Prompt AI: "Bagaimana cara mengambil koordinat GPS presisi tinggi di Flutter? Buat service yang mengambil lat/long akurat < 5 meter, dan mengubahnya menjadi format GeoJSON Point untuk dikirim ke API."
- Eksekusi Manusia: Pastikan permission lokasi di `AndroidManifest.xml` aktif. Lakukan uji coba di luar ruangan (karena GPS indoor tidak akurat).

Step 2: Pengembangan Executive Dashboard (Web)

- Prompt AI: "Buatkan komponen tabel di React.js (atau Vue/Blade sesuai pilihan Anda) untuk menampilkan daftar TPH yang memiliki potensi restan. Tambahkan kolom Status (Normal/Delayed) berdasarkan field `delivery_time`."
- Eksekusi Manusia: Lakukan code review. AI sering membuat UI yang berantakan (AI slop UI). Perbaiki styling secara manual agar sesuai standar dashboard kebun.

FASE 4: PENGUJIAN LABORATORIUM & KEAMANAN

Step 1: Implementasi Enkripsi AES-256

- Prompt AI: "Berikan contoh kode Dart untuk mengenkripsi string JSON menggunakan paket `encrypt` (AES-256 CBC mode) sebelum menyimpannya di Isar DB, dan cara mendekripsinya saat akan dibaca."
- Eksekusi Manusia: Pastikan key disimpan aman di Flutter Secure Storage, jangan di-hardcode.

Step 2: Simulasi Blankspot & Stress Testing

- Skenario Uji: Matikan Wi-Fi/Seluler -> Input 100 data panen -> Nyalakan koneksi -> Biarkan sync -> Cek database backend apakah ada data yang hilang/konflik.
- Prompt AI (untuk QA Script): "Buatkan script Python untuk melakukan stress testing endpoint API `/sync` dengan 1000 request simultan menggunakan library `locust`. Sertakan header JWT."
- Eksekusi Manusia: Jalankan script, catat *response time* dan *error rate*. Pastikan memenuhi target (Sync Success Rate $\geq 98\%$, Latensi < 5 detik).

FASE 5: UJI COBA LAPANGAN (UAT)

Di tahap ini, kode sudah harus *freeze* (tidak ada penambahan fitur baru).

Step 1: Persiapan Responden & Device

- Siapkan 3 Unit Android Low-spec (sebagai RAB).
- Install APK *release mode* (bukan debug mode, karena debug mode boros RAM).
- Kirimkan panduan singkat (Quick Reference Guide) ke layar HP pengguna.

Step 2: Pelaksanaan UAT & Pengambilan SUS

- Prompt AI: "Buatkan 10 pertanyaan kuesioner System Usability Scale (SUS) dalam Bahasa Indonesia yang mudah dipahami oleh Mandor dan Krani kebun sawit. Sertakan pilihan Likert 1-5."
 - Eksekusi Manusia: Cetak kuesioner. dampingi 20 responden saat menggunakan aplikasi. Catat juga keluhan UX (misal: tombol terlalu kecil, font kurang besar, dll).
-

FASE 6: ANALISIS & PUBLIKASI

Step 1: Analisis Data SUS

- Prompt AI: *"Saya memiliki data mentah dari 20 responden SUS (1-5). Berikan formula manual atau script Excel cara menghitung skor akhir SUS (0-100). Jangan hitungkan nilainya, beri saya caranya saja."* (Menyuruh AI menghitung sering kali menghasilkan halusinasi hitungan, lebih baik minta rumusnya).

Step 2: Drafting Artikel SINTA

- Eksekusi Manusia: Gunakan AI (Claude/GPT) sebagai *proofreader* dan *paraphraser*.
- Prompt AI: *"Tolong paraphrase paragraf ini agar lebih akademis dan sesuai gaya jurnal SINTA bidang Teknologi Informasi: [Masukkan draf]. Pastikan tidak mengubah makna teknis dari algoritma resolusi konflik."*